

```
#include <stdio.h>

#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void push(struct Node** top, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *top;
    *top = newNode;
}

int pop(struct Node** top) {
    if (*top == NULL) {
        printf("Stack Underflow!\n");
        return -1;
    }
    struct Node* temp = *top;
    int popped = temp->data;
    *top = temp->next;
    free(temp);
    return popped;
}
```

```

void displayStack(struct Node* top) {
    if (top == NULL) {
        printf("Stack Empty!\n");
        return;
    }
    printf("Stack: ");
    while (top != NULL) {
        printf("%d -> ", top->data);
        top = top->next;
    }
    printf("NULL\n");
}

```

```

struct Queue {
    struct Node* front;
    struct Node* rear;
};

```

```

void enqueue(struct Queue* q, int data) {
    struct Node* newNode = createNode(data);

    if (q->rear == NULL) {
        q->front = q->rear = newNode;
        return;
    }

    q->rear->next = newNode;
    q->rear = newNode;
}

```

```

int dequeue(struct Queue* q) {

```

```

if (q->front == NULL) {
    printf("Queue Underflow!\n");
    return -1;
}

struct Node* temp = q->front;
int data = temp->data;

q->front = q->front->next;

if (q->front == NULL)
    q->rear = NULL;

free(temp);
return data;
}

void displayQueue(struct Queue* q) {
    struct Node* temp = q->front;
    if (temp == NULL) {
        printf("Queue Empty!\n");
        return;
    }

    printf("Queue: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

```

int main() {

    struct Node* stackTop = NULL;

    struct Queue q;

    q.front = q.rear = NULL;


    int choice, value;


    while (1) {

        printf("\n----- MENU ----- \n");

        printf("1. Push (Stack)\n");
        printf("2. Pop (Stack)\n");
        printf("3. Display Stack\n");
        printf("4. Enqueue (Queue)\n");
        printf("5. Dequeue (Queue)\n");
        printf("6. Display Queue\n");
        printf("7. Exit\n");

        printf("Enter choice: ");

        scanf("%d", &choice);


        switch (choice) {

            case 1:

                printf("Enter value to push: ");

                scanf("%d", &value);

                push(&stackTop, value);

                break;


            case 2:

                value = pop(&stackTop);

                if (value != -1)

                    printf("Popped element = %d\n", value);

```

```
break;
```

```
case 3:
```

```
displayStack(stackTop);
```

```
break;
```

```
case 4:
```

```
printf("Enter value to enqueue: ");
```

```
scanf("%d", &value);
```

```
enqueue(&q, value);
```

```
break;
```

```
case 5:
```

```
value = dequeue(&q);
```

```
if (value != -1)
```

```
printf("Dequeued element = %d\n", value);
```

```
break;
```

```
case 6:
```

```
displayQueue(&q);
```

```
break;
```

```
case 7:
```

```
printf("Exiting...\n");
```

```
exit(0);
```

```
default:
```

```
printf("Invalid choice!\n");
```

```
}
```

```
}
```

return 0;

```
1. Push (Stack)
2. Pop (Stack)
3. Enqueue (Queue)
4. Dequeue (Queue)
5. Display
6. Exit
1
Enter value:
20

1. Push (Stack)
2. Pop (Stack)
3. Enqueue (Queue)
4. Dequeue (Queue)
5. Display
6. Exit
5
20 -> 10 -> NULL

1. Push (Stack)
2. Pop (Stack)
3. Enqueue (Queue)
4. Dequeue (Queue)
5. Display
6. Exit
2
Popped: 20

1. Push (Stack)
2. Pop (Stack)
3. Enqueue (Queue)
4. Dequeue (Queue)
5. Display
6. Exit
3
Enter value:
30

1. Push (Stack)
2. Pop (Stack)
3. Enqueue (Queue)
4. Dequeue (Queue)
5. Display
6. Exit
5
10 -> 30 -> NULL

1. Push (Stack)
2. Pop (Stack)
3. Enqueue (Queue)
4. Dequeue (Queue)
5. Display
6. Exit
4
Dequeued: 10

1. Push (Stack)
2. Pop (Stack)
3. Enqueue (Queue)
4. Dequeue (Queue)
5. Display
6. Exit
}
```